**CUDA_Experiment_8\matrix_multiply.cu**

```cpp
#include <stdio.h>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t err__ = (call); \
    if (err__ != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, cudaGetErrorString(err__)); \
        return 1; \
    } \
} while (0)

__global__ void matrixMultiply(const int *A, const int *B, int *C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        int sum = 0;
        for (int k = 0; k < N; ++k)
            sum += A[row*N+k] * B[k*N+col];
        C[row*N+col] = sum;
    }
}

int main() {
    const int N = 2;
    int h_A[N*N] = {1, 2, 3, 4};
    int h_B[N*N] = {5, 6, 7, 8};
    int h_C[N*N] = {0};
    int *d_A = NULL, *d_B = NULL, *d_C = NULL;

    CUDA_CHECK(cudaMalloc((void**)&d_A, N*N*sizeof(int)));
    CUDA_CHECK(cudaMalloc((void**)&d_B, N*N*sizeof(int)));
    CUDA_CHECK(cudaMalloc((void**)&d_C, N*N*sizeof(int)));
    CUDA_CHECK(cudaMemcpy(d_A, h_A, N*N*sizeof(int), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_B, h_B, N*N*sizeof(int), cudaMemcpyHostToDevice));

    dim3 threads(16, 16);
    dim3 blocks((N + 15) / 16, (N + 15) / 16);
    matrixMultiply<<<blocks, threads>>>(d_A, d_B, d_C, N);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(h_C, d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost));

    printf("Matrix Multiplication Result:\n");
    for (int r = 0; r < N; ++r) {
        for (int c = 0; c < N; ++c) printf("%d ", h_C[r*N+c]);
        printf("\n");
    }

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    return 0;
}

```